

■■■ RELATÓRIO DE AUDITORIA DE SEGURANÇA

Projeto: **ReductorPrompt** — Análise de Vulnerabilidades, Isolamento e Conformidade

Data da Auditoria:	15 de Setembro de 2026
Escopo Auditado:	API REST (FastAPI), MCP Hub (FastMCP), Vector Store (ChromaDB), SQLite Telemetry, Frontend React & Scripts
Versão do Sistema:	ReductorPrompt v2.0.0
Classificação:	DOCUMENTO TÉCNICO DEFENSIVO

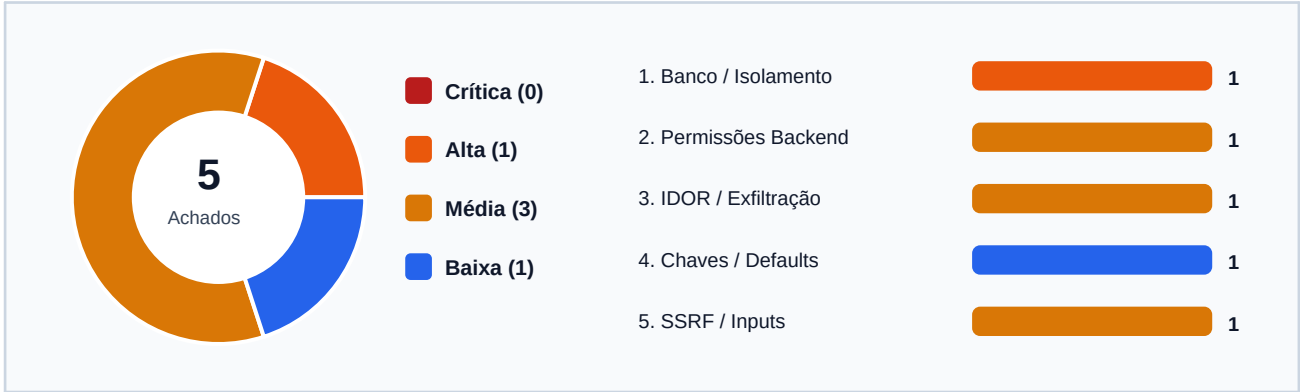
Nota Metodológica & Mapeamento de Stack

Esta auditoria analisou estaticamente 100% dos manipuladores de rota, adaptadores de persistência, comunicação cliente-servidor e rotinas de download do **ReductorPrompt**. A stack detectada e as categorias mapeadas foram:

- **Stack Detectada:** Python 3.11+, FastAPI, ChromaDB (Vector Store), SQLite (Telemetry), FastMCP, React 18 / TypeScript / Vite.
- **1. Banco Sem Tranca:** Mapeado para isolamento no ChromaDB e endpoints de busca/agregação sem filtro de tenant.
- **2. Permissão no Navegador:** Mapeado para operações de controle de VRAM/hardware e ingestão sem verificação de privilégio no backend.
- **3. IDOR:** Mapeado para deleção e exportação de logs e histórico SQLite sem autenticação de posse.
- **4. Chaves Expostas:** Mapeado para passagem de API keys em URLs e CORS wildcard permissivo.
- **5. Inputs Sem Tratamento / SSRF / XSS:** Mapeado para renderização de Markdown no React e downloads dinâmicos de URLs arbitrárias.

1. Resumo Executivo

A auditoria identificou um total de **5 achados acionáveis** e **4 pontos fortes arquiteturais**. O ReductorPrompt apresenta excelente higiene de código contra ataques comuns como **XSS** e **SQL Injection**. No entanto, por ter sido concebido inicialmente como uma ferramenta monousuário de uso local (desktop/localhost), o sistema opera com **ausência total de camadas de autenticação e autorização** em seus endpoints REST, o que representa risco significativo se exposto em rede local compartilhada, Docker bridge ou ambiente em nuvem.



Severidade	Qtd	Impacto Principal
CRÍTICA	0	Nenhuma vulnerabilidade com exploração remota imediata de RCE sem autenticação.
ALTA	1	Ausência de isolamento e autenticação na API REST & ChromaDB em rede aberta.
MÉDIA	3	SSRF no download de URLs; Ações de escrita/VRAM abertas; Exportação irrestrita de dados.
BAIXA	1	API Key do Gemini transitando em query string no fallback HTTP; CORS wildcard.

2. Pontos Fortes e Pontos Fracos

■ Pontos Fortes Verificados (Proteções Ativas)

Prevenção a XSS no Frontend	web/src/.../MarkdownRenderer.tsx: 20-80	Renderizador customizado que converte Markdown em nós nativos React (JSX). Não utiliza dangerouslySetInnerHTML, innerHTML nem eval().
Prevenção a SQL Injection	src/.../system_telemetry_service.py: 157,212	Todas as operações no banco relacional SQLite utilizam queries parametrizadas (?). Nenhuma concatenação direta de string com input do usuário.
Higiene de Segredos em Git	.gitignore:1-10 src/config/settings.py:17	Arquivos .env são estritamente ignorados pelo Git. Chaves padrão utilizam strings vazias e bloqueiam execução se não preenchidas.
Resiliência do Vector Store	src/.../chroma_vector_store.py:28-39	Fallback automático e transparente do Podman HTTP para SQLite local persistente em caso de falha de conexão de rede.

■■ Pontos Fracos e Riscos Centrais

- **Ausência de Autenticação/Autorização (Zero Trust):** A API REST não possui nenhum mecanismo de autenticação (como API Keys, JWT ou mTLS). Qualquer cliente na rede local tem acesso a comandos administrativos e aos dados do banco vetorial.
- **Risco de SSRF em Downloads Dinâmicos:** A rota /api/v1/fetch aceita URLs arbitrárias e realiza requisições sem validação de destino (permitindo sondar endereços locais 127.0.0.1 e metadados de nuvem).
- **CORS Wildcard com Credentials:** A configuração de CORS libera * com allow_credentials=True.

3. Tabela Detalhada de Achados de Segurança

Relação completa de achados verificados diretamente no código-fonte, linha a linha:

Sev.	Categoria & Arquivo:Linha	Descrição da Falha & Impacto
ALTA	1. Banco Sem Tranca / Falta de Auth src/.../fastapi_app.py:85-125 podman-compose.yml:1-13	Todos os endpoints REST e a porta 8001 do ChromaDB estão expostos sem autenticação nem particionamento por usuário/inquilino. Qualquer chamador de rede pode ler o acervo e disparar ingestão massiva.
MÉDIA	2. Permissão Sem Validação Backend src/.../fastapi_app.py:298-320 web/.../LLMControllerView.tsx:1-120	Ações privilegiadas de controle de GPU/VRAM (/api/v1/llm/toggle, /api/v1/llm/unload) e reconfiguração de provedor são executadas pelo backend sem checagem de privilégio de administrador.
MÉDIA	3. IDOR / Exfiltração de Histórico src/.../fastapi_app.py:270-286 src/.../chroma_vector_store.py:138	Endpoints /api/v1/queries/export e /api/v1/logs entregam todo o histórico de consultas e logs do SQLite a qualquer requisitante sem verificação de posse ou identificação.
MÉDIA	4. SSRF em Fetch de Livros src/.../fastapi_app.py:128-145 scripts/fetch_books.py:153-165,245	O endpoint /api/v1/fetch aceita URLs externas arbitrárias e dispara requisições HTTP GET sem validar endereços de rede privada (RFC 1918), permitindo varredura de portas internas e metadados de nuvem.
BAIXA	5. API Key em Query String & CORS src/.../llm_adapters.py:55 src/.../fastapi_app.py:36	No fallback REST da API do Gemini, a GEMINI_API_KEY é transmitida como parâmetro de URL (?key=...) em vez de header HTTP (x-goog-api-key), podendo vazar em logs intermediários.

4. Plano de Recomendações Priorizadas

P1 (Imediato - Alta Prioridade): Implementação de Autenticação via API Key / Bearer Token

Adicionar middleware de segurança no FastAPI (HTTPBearer ou APIKeyHeader) protegendo todos os endpoints /api/v1/*. Em modo desktop/local, vincular o servidor estritamente a 127.0.0.1 e não a 0.0.0.0.

P2 (Curto Prazo - Média Prioridade): Mitigação de SSRF no Download Dinâmico (/api/v1/fetch)

Validar URLs recebidas no fetch_books.py: permitir apenas esquemas https:// e aplicar resolução de DNS para bloquear endereços IP privados (127.0.0.1, 10.0.0.0/8, 192.168.0.0/16, 169.254.169.254).

P3 (Curto Prazo - Média Prioridade): Restrição de Operações Administrativas & IDOR

Separar rotas de consulta (leitura) de rotas administrativas (escrita/GPU/exportação). Exigir chave de administração para endpoints como /api/v1/llm/* e /api/v1/queries/export.

P4 (Médio Prazo - Baixa Prioridade): Ajuste de Headers de API e CORS

Substituir o parâmetro de query string da API Gemini por cabeçalho x-goog-api-key e restringir o allow_origins do CORS para a URL específica do frontend.

5. Issues Acionáveis para o GitHub

Copie e cole os blocos abaixo diretamente nas **Issues do repositório GitHub** para acompanhamento e correção:

--- ISSUE 1 ---

****Título:** [Segurança] Ausência de Autenticação e Isolamento nas Rotas da API REST e ChromaDB**

****Labels:**** security, priority-high, severity-high

Descrição do Problema

A API FastAPI (`src/adapters/inbound/api/fastapi\_app.py`) e a instância do ChromaDB (`podman-compose.yml`) operam sem nenhuma camada de autenticação (API Key, Bearer Token) ou isolamento de inquilino. Qualquer cliente com acesso à porta 8000/8001 pode consultar a base vetorial completa, listar livros e disparar processos pesados de ingestão.

Evidência

- - `src/adapters/inbound/api/fastapi\_app.py:34-40` (CORS wildcard `allow\_origins=["\*"]`)
- - `src/adapters/inbound/api/fastapi\_app.py:85-125` (Rotas `/api/v1/books` e `/api/v1/ingest` sem dependência de segurança)
- - `podman-compose.yml:1-13` (Porta 8001 exposta sem `CHROMA\_SERVER\_AUTH\_CREDENTIALS`)

Impacto

Exposição total de dados vetoriais, histórico de buscas e possibilidade de negação de serviço (DoS) por sobrecarga de ingestão em ambientes compartilhados.

Sugestão de Correção

1. Adicionar middleware de autenticação baseado em `API\_KEY` configurável via `.env`.
2. Vincular o FastAPI e o ChromaDB ao `127.0.0.1` por padrão no `podman-compose.yml` e no `uvicorn.run`.
3. Ativar autenticação básica por token no ChromaDB.

Critérios de Aceite

- [] Todas as rotas `/api/v1/\*` retornam HTTP 401 caso o cabeçalho `X-API-Key` não seja fornecido.
- [] O arquivo `podman-compose.yml` vincula a porta a `127.0.0.1:8001:8000`.
- [] A suíte de testes valida a rejeição de requisições não autorizadas.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

****Título:** [Segurança] Vulnerabilidade de Server-Side Request Forgery (SSRF) no Endpoint /api/v1/fetch******Labels:**** security, priority-medium, severity-medium**Descrição do Problema**

O endpoint `/api/v1/fetch` e a função `scripts/fetch_books.py:ResilientDownloader` recebem URLs fornecidas pelo usuário e realizam downloads HTTP GET usando `urllib.request.urlopen` sem validar o protocolo ou verificar se o IP de destino pertence a redes privadas/locais.

Evidência

- - `src/adapters/inbound/api/fastapi_app.py:128-145`
- - `scripts/fetch_books.py:153-165, 245-250`

```
req = urllib.request.Request(url, headers={"User-Agent": DEFAULT_USER_AGENT})
```

```
with urllib.request.urlopen(req, timeout=self.timeout) as resp:
```

Impacto

Um atacante pode submeter URLs como `http://127.0.0.1:11434/api/tags` ou `http://169.254.169.254/latest/meta-data/` para mapear serviços internos locais, manipular o Ollama ou extrair credenciais de instâncias cloud.

Sugestão de Correção

1. Validar que a URL utiliza estritamente o esquema `https://`.
2. Resolver o DNS do hostname antes da requisição e rejeitar endereços IP pertencentes a:
 - Loopback (`127.0.0.0/8`)
 - Redes Privadas (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`)
 - Link-local / Cloud Metadata (`169.254.0.0/16`)

Critérios de Aceite

- [] URLs com `http://` não seguro são rejeitadas com HTTP 400.
- [] URLs apontando para localhost ou IPs privados são bloqueadas antes de abrir conexão.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---****Título:** [Segurança] Exfiltração de Histórico de Consultas e Logs sem Controle de Acesso (IDOR/Privilege)******Labels:**** security, priority-medium, severity-medium**Descrição do Problema**

Os endpoints de exportação de telemetria (`/api/v1/queries/export`) e logs (`/api/v1/logs`) retornam o banco de dados completo de consultas e mensagens operacionais a qualquer cliente HTTP, sem segregação de usuário ou chave de administrador.

Evidência

- - `src/adapters/inbound/api/fastapi\_app.py:270-286`

```
@app.get("/api/v1/queries/export")
```

```
async def export_queries(format: str = Query(default="json", pattern="^(json|csv)$")):
```

```
content = await asyncio.to_thread(telemetry_service.export_queries, format)
```

Impacto

Vazamento de todas as perguntas realizadas por usuários ou outros agentes no sistema, expondo código-fonte, tópicos confidenciais de projetos e logs de telemetria interna.

Sugestão de Correção

1. Proteger os endpoints `/api/v1/queries/\*`, `/api/v1/logs` e `/api/v1/llm/\*` exigindo perfil/chave administrativa.
2. Limitar o histórico retornado apenas a consultas autorizadas.

Critérios de Aceite

- [] Exportação de queries exige autenticação administrativa.
- [] Logs operacionais só podem ser lidos localmente ou por administradores.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---****Título:** [Segurança] Envio de Chave de API Gemini em Query String na Requisição HTTP Fallback**

****Labels:**** security, priority-low, severity-low

Descrição do Problema

No adaptador `GeminiLLMAdapter`, caso a biblioteca oficial `google-genai` não esteja instalada e o fallback HTTP seja acionado, a `GEMINI\_API\_KEY` é enviada diretamente na URL como parâmetro de consulta (`?key=...`).

Evidência

- - `src/adapters/outbound/llm/llm\_adapters.py:55`

url = f"https://generativelanguage.googleapis.com/v1beta/models/{self.model}:generateContent?key={self.api_key}"

Impacto

Chaves de API em query strings são gravadas em logs de proxies, gateways corporativos, históricos de navegadores e logs de rede, aumentando o risco de comprometimento da credencial.

Sugestão de Correção

Alterar o envio da chave para o cabeçalho HTTP padrão da Google Cloud: `x-goog-api-key: `.

Critérios de Aceite

- [] A URL da requisição de fallback não contém o parâmetro `?key=`.
- [] A chave é passada via header `headers={"x-goog-api-key": self.api\_key}`.

--- FIM ISSUE 4 ---